



ОНЛАЙН-ОБРАЗОВАНИЕ

Проверить включена ли запись?



Меня хорошо слышно && видно?



Напишите в чат, если есть проблемы!

Ставьте ☐ + если все хорошо

Ставьте ☐ - если есть проблемы

Продвинутая конфигурация Spring-приложений



- Профили и автоконфигурации (@Profile, @Conditional)
- Spring boot test

Чуть-чуть:

- Application events
- Spring Shell

Поехали!

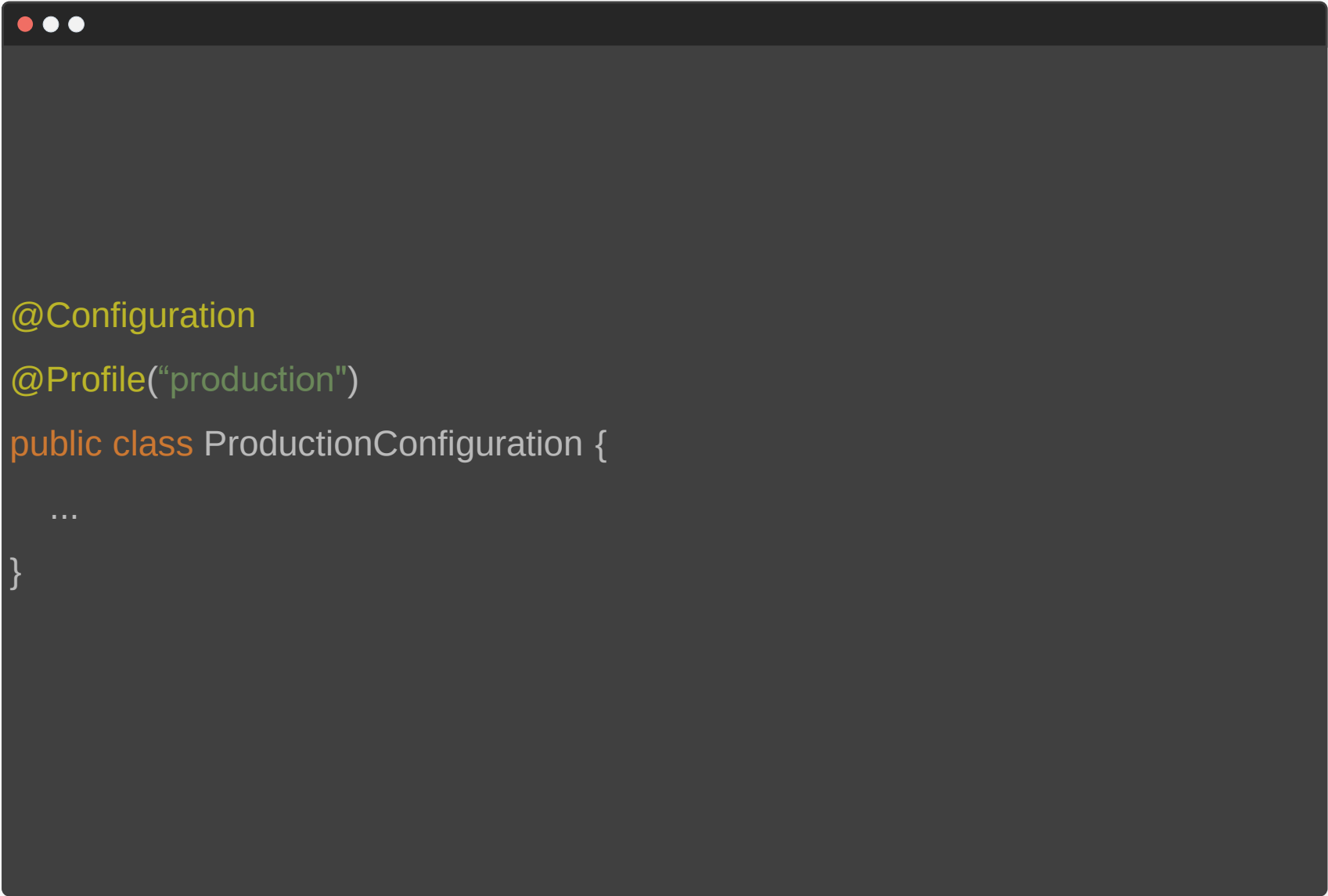
01

Профили и автоконфигурации (@Profile, @Conditional)



- Иногда бывает так, что приложение должно запускаться в разных вариантах
- Иногда это разные БД
- Иногда staging окружения - dev, test, prod, ...


```
server:
  port: 8088
spring:
  profiles:
    active: development
---
spring:
  profiles: development
server:
  port: 8888
---
spring:
  profiles: production
server:
  port: 8080
```



```
@Configuration
@Profile("production")
public class ProductionConfiguration {
    ...
}
```

- Их может быть одновременно несколько
- В зависимости от профиля создаются те или иные бины и конфигурации
- По сути выходит каждый раз другое приложение для каждого набора профилей
- С учетом, что у нас теперь есть докер и возможность переопределять настройки через переменные окружения, профили не самый лучший для применения инструмент

- @Conditional* это мощная возможность Spring Boot. Auto-конфигурация («магия» Spring Boot) базируется на ней
- С @Conditional* Вы можете создавать бины:
 - Если свойство не/задано;
 - Есть класс в classpath;
 - Кастомные условия;
 - ...

```
@Bean
@ConditionalOnProperty(value = "create.socket", havingValue = "always")
@ConditionalOnClass(name = "OtherService")
@Conditional(CustomCondition.class)
public AnyService anyService(){
    ...
}
```

Упражнение №1

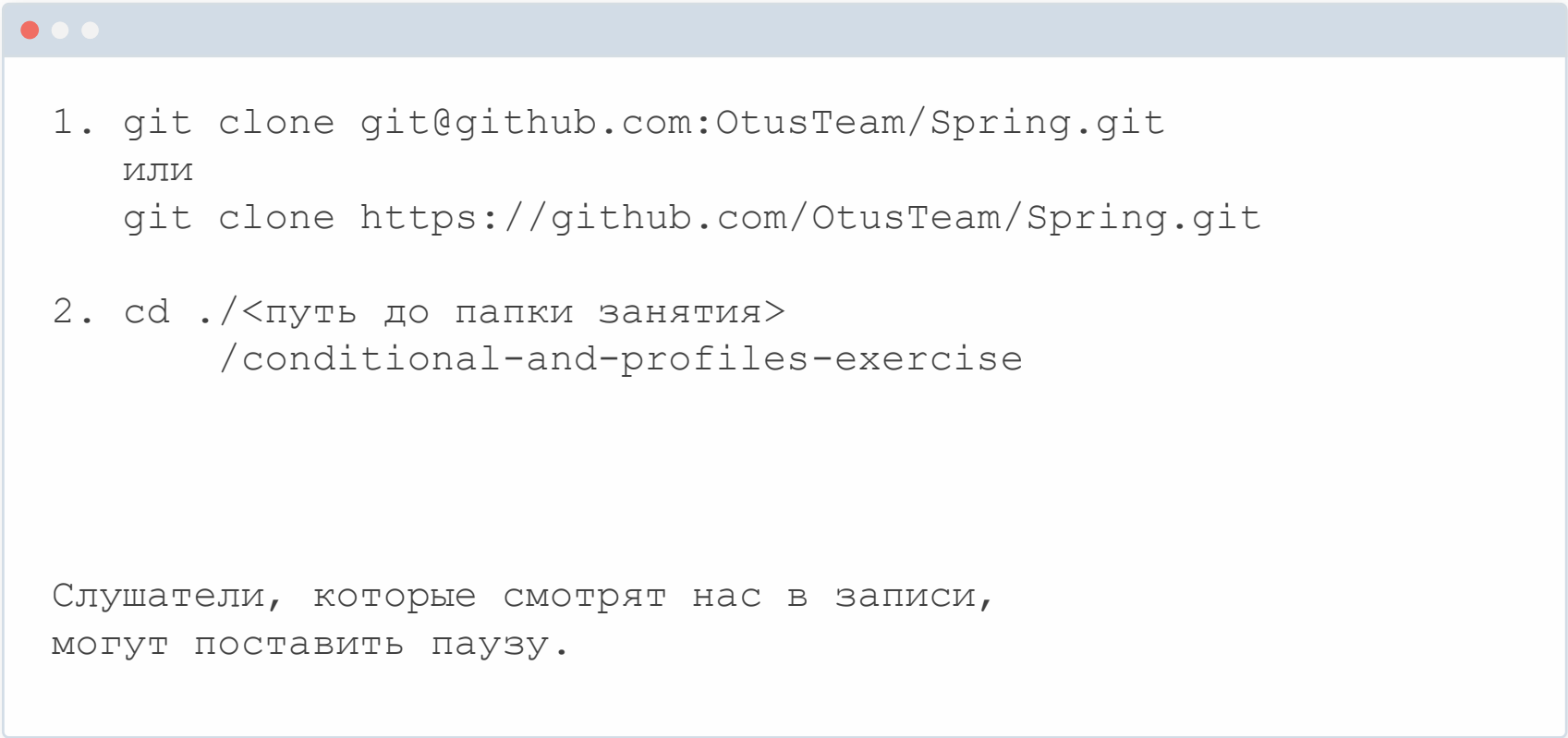
(conditional-and-profiles-exercise)



Условия вечеринки:

- Алексей придет если `condition.alexey-exists=true`
- Анна придет если будет Алексей
- Олег придет если включен профиль Oleg
- Петр придет если включен профиль Peter
- Янис придет если `condition.yanis-exists=true`
- Яна придет если нет Алексея, но есть Янис

Измените только одну настройку в `application.yml`, чтобы на вечеринку пришло максимальное количество людей

A screenshot of a terminal window with a light blue header bar containing three window control buttons (red, yellow, green). The terminal has a white background and displays two numbered steps for cloning a repository and navigating to a directory.

```
1. git clone git@github.com:OtusTeam/Spring.git
   или
   git clone https://github.com/OtusTeam/Spring.git

2. cd ./<путь до папки занятия>
   /conditional-and-profiles-exercise
```

Слушатели, которые смотрят нас в записи,
могут поставить паузу.

- Для формирования контекста в интеграционных тестах. Для этого есть то, что мы будем изучать сегодня чуть позже
- Для загрузки свойств из нужного файла в интеграционных тестах. `application.yml` в тестовых ресурсах переопределит основной
- `@ActiveProfile` нужен для тестирования профиля приложения, а не для включения профиля тестирования

Вопросы?

(ставим “+” если вопросы есть и “-”
если вопросов нет)

02

События (ApplicationEvents)



- Нечто вроде широкопередаточной сирены по всем бинам, кто умеет её слушать
- Есть много событий самого фреймворка (`ApplicationContextInitializedEvent`)
- Но можно делать свои собственные
- События – наследники `ApplicationEvent`
- `ApplicationEventPublisher` – публикует события
- Чтобы слушать событие бину нужно реализовать интерфейс `ApplicationListener` или повесить над хендлером сообщения `@EventListener`

```
public class CustomEvent extends ApplicationEvent {  
    private final String message;  
  
    public CustomEvent(Object source, String message) {  
        super(source);  
        this.message = message;  
    }  
  
    public String getMessage() {  
        return message;  
    }  
}
```

```
public class CustomEventPublisher {  
    private final ApplicationEventPublisher publisher;  
  
    @Autowired  
    public CustomEventPublisher(ApplicationEventPublisher publisher) {  
        this.publisher = publisher;  
    }  
  
    public void doStuffAndPublishAnEvent(final String message) {  
        System.out.println("Publishing custom event...");  
        CustomEvent evt = new CustomEvent(this, message);  
        publisher.publishEvent(evt);  
    }  
}
```

```
public class CustomEventListener implements ApplicationListener<CustomEvent> {  
    @Override  
    public void onApplicationEvent(CustomEvent evt) {  
        System.out.println("Received custom event: " + evt.getMessage());  
    }  
}  
  
public class CustomEventListener {  
    @EventListener  
    public void onApplicationEvent(CustomEvent evt) {  
        System.out.println("Received custom event: " + evt.getMessage());  
    }  
}
```

03

Spring Shell



- Отличный вариант нормального консольного интерфейса
- `@ShellComponent` – набор методов-команд
- `@ShellMethod` – метод-команда
- `@ShellOption` – аргумент (передается как `--name`, обязательна по умолчанию в 2.0.0)
- `@ShellMethodAvailability` – позволяет управлять доступностью команды


```
<dependencies>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.shell</groupId>
    <artifactId>spring-shell-starter</artifactId>
  </dependency>
</dependencies>
```

```
@ShellComponent
public class HelloCommands {
    private final HelloService helloService;

    @Autowired
    public void HelloCommands (HelloService helloService) {
        this.helloService = helloService;
    }

    @ShellMethod("Hello")
    public String hello (@ShellOption String name) {
        return helloService.hello(name);
    }
}
```

Демо (Spring Shell и ApplicationEvents)



Вопросы?

(ставим “+” если вопросы есть и “-”
если вопросов нет)

04

Конфигурация тестов (@SpringBootTest, ...)



- `@SpringBootTest` – позволяет писать тесты с поднятием контекста `SpringBoot`, включает `@MockBean`
- `@RunWith(SpringRunner.class)`,
`@ExtendWith({SpringExtension.class})` – расширения для запуска тестов для JUnit4 и JUnit5 соответственно. Без них тесты не стартуют)
- `@MockitoBean` (ранее `@MockBean`) – создаёт мок (Mockito), который перед каждым тестом заменяет исходный бин в контексте

```
@RunWith(SpringRunner.class) // Писать если используем JUnit4
@ExtendWith(SpringExtension.class) // Писать если используем JUnit5
@SpringBootTest
public class AnyServiceTest {

    @MockitoBean
    private AnyDependency anyDependency;

    @Autowired
    private Service service;

    @Test
    public void exampleTest () {
        given(anyDependency.someCall()).willReturn("mock");
        assertTrue(service.get())
    }
}
```

- Эта аннотация ставится на классе
- Поднимается вверх по пакетам к самому корню (напомню классы в “src/test/java” лежат вместе с src/main/java)
- Ищет класс @SpringBootConfiguration (@SpringBootApplication) и обрабатывает её
- Спускается в глубь пакетов (на @SpringBootApplication есть @ComponentScan) и находит все конфигурации/бины
- И поднимает все, КРОМЕ ТЕХ, для которых указан @MockitoBean
- Если есть БД, то её и поднимет

- Если разместить класс аннотированный @SpringBootTestConfiguration на пути сканирования (например, в “src/test/java/ru.otus.services/”), но до Main-класса, то поиск остановится
- И обработается данный конфиг и все что над ним висит

- Если разместить внутренний класс @Configuration внутри вашего теста (класс внутри класса)
- То будет использоваться он вместо конфигурации приложения, и поиск вообще не будет производиться
- Того же эффекта можно добиться если воспользоваться ограничением контекста через @ContextConfiguration или через аргумент classes самого @SpringBootTest

```
@SpringBootTest
public class AnyServiceTest {

    @Configuration
    static class TestConfiguration {
        @Bean
        public Service service () {
            return new ServiceImpl();
        }
    }

    @MockitoBean
    private AnyDependency anyDependency;

    @Autowired
    private Service service;

    // Write tests here
}
```

```
@SpringBootTest
@ContextConfiguration(classes = Service.class)
public class AnyServiceTest {

    @MockitoBean
    private AnyDependency anyDependency;

    @Autowired
    private Service service;

    // Write tests here
}
```

```
@SpringBootTest (classes = Service.class)
public class AnyServiceTest {

    @MockitoBean
    private AnyDependency anyDependency;

    @Autowired
    private Service service;

    // Write tests here
}
```

- Если разместить внутренний класс @TestConfiguration внутри вашего теста (класс внутри класса)
- То будет использоваться эта конфигурация + КОНФИГУРАЦИЯ в @SpringBootApplication

```
@SpringBootTest
public class AnyServiceTest {

    @TestConfiguration
    static class TestConfiguration {
        @Bean
        public Service service () {
            return new ServiceNewImpl();
        }
    }

    @MockitoBean
    private AnyDependency anyDependency;

    @Autowired
    private Service service;

    // Write tests here
}
```

Упражнение №2

(test-configuration-exercise-1)



С помощью вложенных конфигураций сделать так, чтобы прошел `NestedConfigurationDemoTest` (family должно содержать только собаку)

- ✓ Сделать вложенный класс конфигурации. Не забываем про `static`
- ✓ Выбрать тип конфигурации (`@Configuration/@TestConfiguration`)
- ✓ С помощью `@Bean/@ComponentScan` добавить в контекст нужный бин(ы)

```
1. git clone git@github.com:OtusTeam/Spring.git
   или
   git clone https://github.com/OtusTeam/Spring.git
```

```
2. cd /<путь до папки занятия>
   /test-configuration-exercise-1
```

Или можно сразу посмотреть решение в проекте
`test-configuration-solution-1`

Слушатели, которые смотрят нас в записи, могут поставить паузу.

Упражнение №3

(test-configuration-exercise-2)



С помощью вложенных конфигураций сделать так, чтобы прошел `NestedTestConfigurationDemoTest` (family должно содержать маму, папу, сына и собаку)

- ✓ Сделать вложенный класс конфигурации. Не забываем про `static`
- ✓ Выбрать тип конфигурации (`@Configuration/@TestConfiguration`)
- ✓ С помощью `@Bean/@ComponentScan` добавить в контекст нужный бин(ы)

```
1. git clone git@github.com:OtusTeam/Spring.git
   или
   git clone https://github.com/OtusTeam/Spring.git
```

```
2. cd /<путь до папки занятия>
   /test-configuration-exercise-2
```

Или можно сразу посмотреть решение в проекте `test-configuration-solution-2`

Слушатели, которые смотрят нас в записи, могут поставить паузу.

Упражнение №4

(test-configuration-exercise-3)



С помощью внешней конфигурации сделать так, чтобы прошел `SpringBootTestWithExternalLimitationDemoTest` (family должно содержать маму, папу и сына)

- ✓ Над `TestSpringBootConfiguration` повесить `@SpringBootTestConfiguration`
- ✓ С помощью `@Bean/@ComponentScan` добавить в контекст нужный бин(ы)

```
1. git clone git@github.com:OtusTeam/Spring.git
   или
   git clone https://github.com/OtusTeam/Spring.git

2. cd /<путь до папки занятия>
   /test-configuration-exercise-3
```

Или можно сразу посмотреть решение в проекте `test-configuration-solution-3`

Слушатели, которые смотрят нас в записи, могут поставить паузу.

Вопросы?

(ставим “+” если вопросы есть и “-”
если вопросов нет)

Демо

(Кэширование контекста)

Вопросы?

(ставим “+” если вопросы есть и “-”
если вопросов нет)

05

Домашнее задание



- Переписать интерфейс приложения для тестирования студентов с использованием Spring Shell.
- Решение должно удовлетворять нижеуказанным требованиям.

- Учесть актуальные требования прошлой работы
- Подключить Spring Shell, используя стартер
- Написать команду/набор команд, позволяющий проводить тестирование (можно сделать одну команду, запускающую тестирование)
- Учесть, что классы с командами Spring Shell не являются сервисами, они по сути на следующем слое (в т.ч. это касается имен классов, пакетов и аннотаций над классами)
- Перенести тесты на `@SpringBootTest`. Компоненты и моки взять из тестового контекста, в остальном, за исключением интеграции с файловой системой в тесте дао, это должны быть юнит-тесты
- Учесть, что Spring Shell в тестах нужно отключить
- Локализовывать команды Spring Shell не нужно, а сообщения которые они выводят, нужно
- Данная работа не подразумевает использования Spring Application Events. Особенно это касается написания на них бизнес логики

- Задание сдаётся только после приема предыдущей работы
- Задание сдаётся в виде ссылки на pull-request в чат с преподавателем в личном кабинете ОТУС, не в Telegram
- В pull-request должно присутствовать только то что касается текущей работы. Временные файлы и файлы IDE не должны попадать в PR.
- Инструкция по работе с pull-request-ами находится в документе "Настройка окружения.pdf" в материалах занятия.
- Вопросы можно задавать в чате ДЗ, но для оперативности рекомендуем делать это в группе Telegram.
- Код, написанный в данном ДЗ больше использоваться не будет

Пожалуйста, пройдите опрос

Спасибо за внимание!

Spring ShellA Вам!

